



**INSTITUTO TECNOLÓGICO DE CULIACÁN**

**Materia:**

**Tópicos de Inteligencia Artificial**

**Validación de tarea: AG**

**Integrantes:**

**Robles Zamora Anette Leticia**

**Ruelas Gutiérrez Maia Paulina**

**Maestro:**

**Mora Félix Zuriel Dathan**

**08/11/2025**

## Introducción:

El presente documento detalla la implementación y validación de un algoritmo genético como parte de la Tarea de Validación III. El objetivo principal de este proyecto es aplicar los conceptos teóricos de los algoritmos evolutivos para resolver una versión simplificada del Problema del Viajante (TSP), buscando la ruta más corta que conecte una serie de puntos geográficos. La solución se desarrolló en Python, siguiendo los componentes clave de un algoritmo genético: se genera una población inicial de rutas aleatorias, se evalúa la aptitud de cada una en función de su distancia total, se seleccionan los individuos más prometedores y se generan nuevas soluciones mediante operadores de cruce y mutación. Este proceso iterativo permite que la población evolucione hacia soluciones de mayor calidad a lo largo de las generaciones. Para facilitar la revisión y validación del trabajo realizado, se anexa la siguiente estructura de archivos:

**Código Fuente (AG.py):** El script principal de Python, modularizado y debidamente comentado, que contiene la implementación completa del algoritmo. **Documentación (README.md):** Un archivo que proporciona una descripción general del proyecto, las dependencias necesarias y las instrucciones detalladas para su ejecución.

**Código de Pruebas (Pruebas\_unitarias.py):** Un script de pruebas formales diseñado para verificar el correcto funcionamiento de los componentes críticos del algoritmo, como el cálculo de aptitud, el cruce y la mutación. A continuación, se presentan las capturas de pantalla que demuestran los resultados obtenidos al ejecutar tanto el programa principal como el script de pruebas, validando así la funcionalidad y la efectividad de la implementación.

## 1-Código Python reconstruido:

```
import random
import numpy as np
import operator

# representa cada municipio o ciudad con sus coordenadas
class Municipio:

    #representa una ciudad en el mapa con coordenadas (x, y)
```

```

def __init__(self, x, y):
    self.x = x
    self.y = y

def distancia(self, otro_municipio):
    """
    Calcula la distancia euclidiana (en línea recta) a otro municipio.
    Usa el Teorema de Pitágoras.
    """
    distancia_x = abs(self.x - otro_municipio.x)
    distancia_y = abs(self.y - otro_municipio.y)
    distancia = np.sqrt(distancia_x**2 + distancia_y**2)
    return distancia

def __repr__(self):
    """
    Representación en texto de un objeto Municipio, útil para imprimir.
    """
    return f"({self.x},{self.y})"

# funcion de aptitud (Fitness)

class Aptitud:

    def __init__(self, ruta):
        self.ruta = ruta
        self.distancia_total = 0
        self.valor_aptitud = 0.0

    def calcular_distancia(self):

        if self.distancia_total == 0:
            distancia_parcial = 0
            for i in range(len(self.ruta)):
                punto_origen = self.ruta[i]
                # Si no es la última ciudad, el destino es la siguiente
                if i + 1 < len(self.ruta):
                    punto_destino = self.ruta[i + 1]
                # Si es la última ciudad, el destino es la primera (ruta
circular)
                else:
                    punto_destino = self.ruta[0]
                distancia_parcial += punto_origen.distancia(punto_destino)
            self.distancia_total = distancia_parcial
        return self.distancia_total

```

```

def calcular_aptitud(self):

    if self.valor_aptitud == 0:
        self.valor_aptitud = 1 / float(self.calcular_distancia())
    return self.valor_aptitud

# inicializacion de la población

def crear_ruta_inicial(lista_municipios):

    #Crea una permutación de la lista de municipios

    ruta = random.sample(lista_municipios, len(lista_municipios))
    return ruta

def crear_poblacion_inicial(tamano_poblacion, lista_municipios):

    poblacion = []
    for _ in range(tamano_poblacion):
        poblacion.append(crear_ruta_inicial(lista_municipios))
    return poblacion

# proceso de seleccion

def clasificar_rutas(poblacion):

    #evalúa la aptitud de cada ruta en la población y las ordena de la mejor a
    la peor.

    resultados_aptitud = {}
    for i in range(len(poblacion)):
        resultados_aptitud[i] = Aptitud(poblacion[i]).calcular_aptitud()

    # Ordena el diccionario por valor de aptitud (el segundo elemento de la
    tupla)
    return sorted(resultados_aptitud.items(), key=operator.itemgetter(1),
reverse=True)

def seleccion(poblacion_clasificada, num_seleccionados_elite):

    resultados_seleccion = []

```

```

# los mejores individuos se aseguran un puesto.
for i in range(num_seleccionados_elite):
    resultados_seleccion.append(poblacion_clasificada[i][0])

# Calculamos la probabilidad acumulada de selección para cada individuo
aptitudes = [item[1] for item in poblacion_clasificada]
suma_total_aptitud = sum(aptitudes)
prob_acumulada = [sum(aptitudes[:i+1]) / suma_total_aptitud for i in
range(len(aptitudes))]

# Selecciona el resto de los individuos
for _ in range(len(poblacion_clasificada) - num_seleccionados_elite):
    num_aleatorio = random.random()
    for i in range(len(poblacion_clasificada)):
        if num_aleatorio <= prob_acumulada[i]:
            resultados_seleccion.append(poblacion_clasificada[i][0])
            break

return resultados_seleccion

def obtener_grupo_apareamiento(poblacion, resultados_seleccion):

    grupo = []
    for indice in resultados_seleccion:
        grupo.append(poblacion[indice])
    return grupo

#cruce y mutacion

def cruce(padre1, padre2):

    hijo_parcial1 = []

    # Selecciona dos puntos aleatorios para el segmento del padre1
    genA = int(random.random() * len(padre1))
    genB = int(random.random() * len(padre1))

    inicio_gen = min(genA, genB)
    fin_gen = max(genA, genB)

    # Copia el segmento del padre1 al hijo
    for i in range(inicio_gen, fin_gen):
        hijo_parcial1.append(padre1[i])

```

```

# Rellena con los genes del padre2 que no estén ya en el hijo
hijo_parcial2 = [gen for gen in padre2 if gen not in hijo_parcial1]

hijo = hijo_parcial1 + hijo_parcial2
return hijo

def reproducir_poblacion(grupo_apareamiento, num_seleccionados_elite):

    hijos = []

    # Mezcla el grupo para que el cruce sea aleatorio
    grupo_mezclado = random.sample(grupo_apareamiento, len(grupo_apareamiento))

    # Conserva a los individuos de élite
    for i in range(num_seleccionados_elite):
        hijos.append(grupo_apareamiento[i])

    # Cruza el resto para llenar la población
    for i in range(len(grupo_apareamiento) - num_seleccionados_elite):
        hijo = cruce(grupo_mezclado[i], grupo_mezclado[len(grupo_apareamiento) -
1 - i])
        hijos.append(hijo)

    return hijos

def mutacion(individuo, tasa_mutacion):

    #aplica una mutación de "intercambio" (Swap)
    #Por cada gen (ciudad) en el individuo, hay una pequeña probabilidad
    #(tasa_mutacion) de que se intercambie con otro gen aleatorio

    for indice_ciudad in range(len(individuo)):
        if random.random() < tasa_mutacion:
            # Elige otra ciudad aleatoria para intercambiar
            indice_intercambio = int(random.random() * len(individuo))

            # Realiza el intercambio
            ciudad1 = individuo[indice_ciudad]
            ciudad2 = individuo[indice_intercambio]
            individuo[indice_ciudad] = ciudad2
            individuo[indice_intercambio] = ciudad1

    return individuo

```

```

def mutar_poblacion(poblacion, tasa_mutacion):

    #Aplica la mutación a toda la población
    poblacion_mutada = []
    for individuo in poblacion:
        poblacion_mutada.append(mutacion(individuo, tasa_mutacion))
    return poblacion_mutada

# bucle principal del AG

def obtener_siguiete_generacion(generacion_actual, num_elite, tasa_mutacion):

    #1-Clasifica la generación actual
    #2-Selecciona a los padres
    #3-Realiza el cruce para crear hijos
    #4-Aplica mutaciones

    pob_clasificada = clasificar_rutas(generacion_actual)
    resultados_sel = seleccion(pob_clasificada, num_elite)
    grupo_ap = obtener_grupo_apareamiento(generacion_actual, resultados_sel)
    hijos = reproducir_poblacion(grupo_ap, num_elite)
    siguiente_generacion = mutar_poblacion(hijos, tasa_mutacion)
    return siguiente_generacion

def algoritmo_genetico(lista_municipios, tam_pob, num_elite, tasa_mut,
num_generaciones):

    # crea la población inicial
    poblacion = crear_poblacion_inicial(tam_pob, lista_municipios)

    # obtiene la distancia inicial para comparar
    distancia_inicial = 1 / clasificar_rutas(poblacion)[0][1]
    print(f"Distancia inicial: {distancia_inicial}")

    # evoluciona la población a lo largo de las generaciones
    for i in range(num_generaciones):
        poblacion = obtener_siguiete_generacion(poblacion, num_elite, tasa_mut)

    # obtiene el resultado final
    clasificacion_final = clasificar_rutas(poblacion)
    distancia_final = 1 / clasificacion_final[0][1]
    print(f"Distancia final: {distancia_final}")

    # devuelve la mejor ruta encontrada
    indice_mejor_ruta = clasificacion_final[0][0]

```

```

    mejor_ruta = poblacion[indice_mejor_ruta]
    return mejor_ruta

# ejecucion del programa

if __name__ == '__main__':
    # lista de municipios
    lista_de_municipios = [
        Municipio(x=40.4168, y=-3.7038), # Madrid
        Municipio(x=41.3874, y=2.1686),  # Barcelona
        Municipio(x=39.4699, y=-0.3763), # Valencia
        Municipio(x=37.3891, y=-5.9845), # Sevilla
        Municipio(x=41.6488, y=-0.8891), # Zaragoza
        Municipio(x=36.7213, y=-4.4214), # Málaga
    ]

    # parámetros del algoritmo genético
    TAMANO_POBLACION = 50
    NUMERO_ELITE = 10
    TASA_MUTACION = 0.01
    NUMERO_GENERACIONES = 100

    # ejecuta el algoritmo
    mejor_ruta = algoritmo_genetico(
        lista_municipios=lista_de_municipios,
        tam_pob=TAMANO_POBLACION,
        num_elite=NUMERO_ELITE,
        tasa_mut=TASA_MUTACION,
        num_generaciones=NUMERO_GENERACIONES
    )

    print("\nMejor ruta encontrada:")
    print(mejor_ruta)

```



## 2-El archivo README con documentación

### # Algoritmo Genético para el Problema del Viajante (TSP)

#### ## Descripción del Proyecto

Este proyecto es una implementación en Python de un algoritmo genético diseñado para resolver una versión simplificada del Problema del Viajante (TSP). El objetivo es encontrar la ruta más corta posible que visite un conjunto de "municipios" (representados por coordenadas x, y) y regrese al punto de partida.

El algoritmo funciona siguiendo los principios de la evolución natural:

1. **\*\*Población Inicial\*\***: Se crea un conjunto de rutas aleatorias.
2. **\*\*Evaluación (Aptitud)\*\***: Cada ruta se evalúa para determinar su "aptitud". En este caso, la aptitud es inversamente proporcional a la distancia total de la ruta (rutas más cortas son más aptas).
3. **\*\*Selección\*\***: Las rutas más aptas son seleccionadas para crear la siguiente generación. Se utiliza una técnica de "elitismo" para asegurar que las mejores soluciones sobrevivan.
4. **\*\*Cruce (Reproducción)\*\***: Las rutas seleccionadas se combinan para crear nuevas rutas "hijo", heredando características de sus "padres".
5. **\*\*Mutación\*\***: Se introducen pequeños cambios aleatorios en las nuevas rutas para mantener la diversidad genética y evitar estancarse en soluciones subóptimas.

Este proceso se repite durante un número determinado de generaciones, con la esperanza de que la población evolucione hacia una solución óptima o casi óptima.

#### ## Instrucciones de Ejecución

Para ejecutar este código, sigue estos pasos:

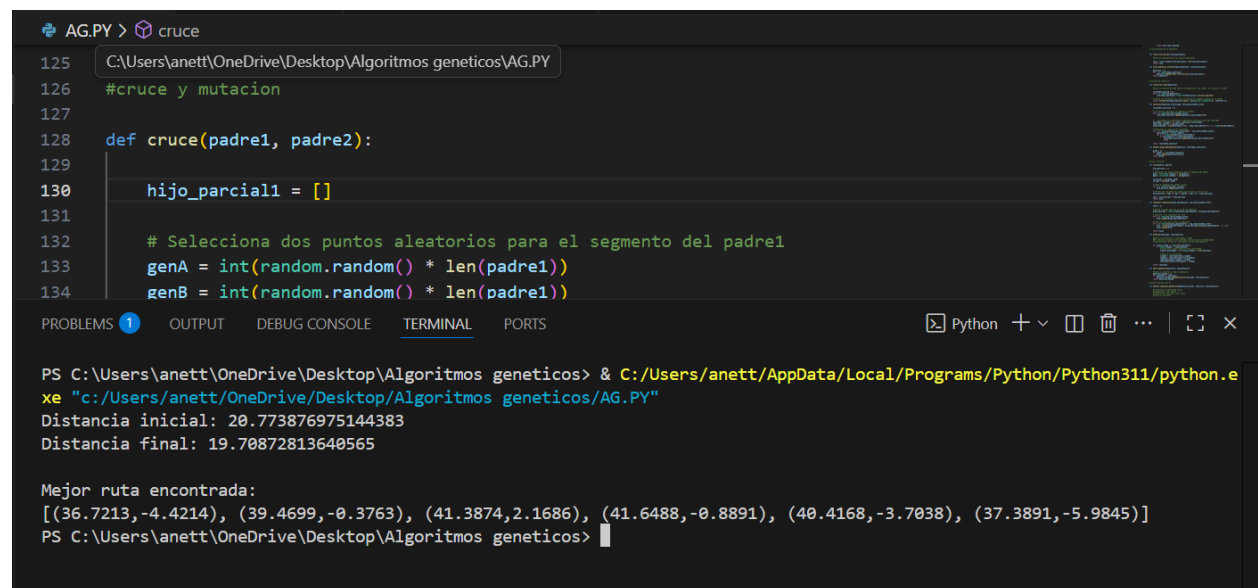
1. **\*\*Asegúrate de tener Python instalado\*\*** en tu sistema.
2. **\*\*Instala las dependencias necesarias\*\***. Este proyecto requiere la biblioteca ``numpy``. Puedes instalarla usando pip:  
``bash  
pip install numpy  
``
3. **\*\*Guarda el código\*\*** en un archivo llamado ``algoritmo_genetico.py``.
4. **\*\*Ejecuta el script\*\*** desde tu terminal:  
``bash  
python algoritmo\_genetico.py  
``

El script imprimirá la distancia de la mejor ruta en la población inicial y la distancia de la mejor ruta encontrada después de todas las generaciones. Finalmente, mostrará la secuencia de coordenadas de la mejor ruta.

## ## Dependencias

- Python 3.x
- `numpy`

## 3-Cualquier caso de prueba y resultados que haya generado



The screenshot shows a VS Code editor with a Python file named `AG.PY` open. The file contains a function `cruce` for a genetic algorithm. The terminal window at the bottom shows the command to run the script and its output.

```
AG.PY > cruce
125 C:\Users\anett\OneDrive\Desktop\Algoritmos geneticos\AG.PY
126 #cruce y mutacion
127
128 def cruce(padre1, padre2):
129
130     hijo_parcial1 = []
131
132     # Selecciona dos puntos aleatorios para el segmento del padre1
133     genA = int(random.random() * len(padre1))
134     genB = int(random.random() * len(padre1))

PS C:\Users\anett\OneDrive\Desktop\Algoritmos geneticos> & C:/Users/anett/AppData/Local/Programs/Python/Python311/python.exe "c:/Users/anett/OneDrive/Desktop/Algoritmos geneticos/AG.PY"
Distancia inicial: 20.773876975144383
Distancia final: 19.70872813640565

Mejor ruta encontrada:
[(36.7213,-4.4214), (39.4699,-0.3763), (41.3874,2.1686), (41.6488,-0.8891), (40.4168,-3.7038), (37.3891,-5.9845)]
PS C:\Users\anett\OneDrive\Desktop\Algoritmos geneticos>
```

Pruebas\_unitarias.py > ejecutar\_pruebas

```
4 def ejecutar_pruebas():  
20     distancia_calculada = aptitud_ruta.calcular_distancia()  
21  
22     print(f"Ruta de prueba: {ruta_prueba}")  
23     print(f"Distancia esperada: {distancia_esperada}")  
24     print(f"Distancia calculada: {distancia_calculada}")  
25  
26     # Verificamos si el resultado es correcto
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

[Prueba 1: Cálculo de Aptitud]

Ruta de prueba: [(0,0), (3,0), (3,4)]

Distancia esperada: 12

Distancia calculada: 12.0

Resultado: PASÓ

[Prueba 2: Validez del Hijo tras el Cruce]

Padre 1: [(0,0), (3,0), (3,4)]

Padre 2: [(3,4), (3,0), (0,0)]

Hijo resultante: [(0,0), (3,0), (3,4)]

Resultado: PASÓ (El hijo es una ruta válida)

[Prueba 3: Validez de la Ruta tras la Mutación]

Ruta original: [(0,0), (3,0), (3,4)]

Ruta mutada: [(0,0), (3,0), (3,4)]

Resultado: PASÓ (La ruta mutada es válida)

--- PRUEBAS FINALIZADAS ---

PS C:\Users\anett\OneDrive\Desktop\Algoritmos geneticos> █